

IPD: Iterated Profile Diagrams

DEFINITIONS & PROOFS / ENGLISH EDITION

2026-09-14, zero-start version v0.1.

Files: NER expander · standalone Python definition · release validation.

An IPD column contains edges to earlier columns. Each edge carries a tree profile, and a tree's head can itself be a tree one level lower. Every ancestral reference inside these nested heads moves during expansion. The notation is not an operation history wrapped around an older system.

The actual column order on the standard reachable domain has a complete paper well-ordering proof in KP_ω + there exists an uncountable ordinal, with full set induction. Its ordinary Lean proof has also been checked. See the paper, definition correspondence, and Lean instructions. The weak metatheory itself is not encoded in Lean.

Comparisons with wY (omega-Y weak magma), TPD, and the other notations in this repository remain unproved. Experimental encoding routes are not presented as release theorems.

1. Using the NER expander

Load the entire `IPD.ne-rewritten.js` file into the custom-notation facility of [ne-rewritten](#). The registration script is standalone: it needs no other custom notation, network request, or Python file.

The default display is 列表 (list). The equivalent-display menu additionally offers 计数序列 (count sequence) and 树形图 (tree view), without a duplicate list button. Displays inherit the application's font. The tree view uses HTML; plain-text and LaTeX modes fall back to the complete list. Example inputs:

```
Limit
A0
A1[6]
A2[3]
Limit[2][3]
[][0:2/.]
```

T_n is an alias for A_n . Evaluating aliases stores real column structures, not their operation histories. A natural number such as 3 means `[][[]]`, not an arbitrary complex column whose count is three. Count sequences are display-only; no general inverse parser is claimed.

For example, A_2 has the following list and count sequence:

```
[][0:2/.]    1,4
```

Its third fundamental-sequence term is:

```
[][0:1/.][1:1/^[2:1/^(.)]
1,3,11,44
```

Each `[]` is one column, read left to right; semicolons separate edges. In an edge $p:d/t$, p is the parent column, d the profile level, and t its tree. Column indices start at zero.

Syntax	Meaning
[]	An empty column, with no edges
∅	The diagram with no columns
.	The independent zero tree at a positive level; the edge still exists
*	SELF: the column containing this edge
^	CAP: the fixed greatest head at the current tree level, not the external TOP
h(t,u)	A node with head h and the ordered children t, u
{t}	A non-CAP head at a higher level, containing the tree t one level lower

Thus $^{\wedge}(\cdot, \cdot, \cdot, \cdot)$ has a CAP head and four zero children. In the level-two profile $2/\{*(\cdot)\}(\cdot, \cdot)$, the head is the level-one tree $*(\cdot)$, and the node has two level-two zero children. Levels are not array dimensions or ordinal exponents. Repeated subtrees are written in full in the list view.

The tree view arranges columns from left to right. Each edge's $\rightarrow p$ gives its parent and level its level. Ordinary ordered children are connected downwards by solid lines. A composite head's entire lower-level tree is nested inside a purple frame labelled with that head's level. Repeated occurrences are drawn separately. Empty columns use $\emptyset$, while zero profiles use a dot. Exact counts appear above the diagram; an unfinished count does not prevent tree rendering. A drawing that exceeds its guard fails explicitly instead of showing a partial tree.

2. Finite trees and comparison

An edge in column j with parent p uses the alphabet

$$X_{p,j} = \{\text{ROOT } 0 < \dots < \text{ROOT } p < \text{SELF } j\}, \quad 0 \leq p < j.$$

Put $K_0(X) = X$. For $d > 0$, $K_d(X)$ contains an independent least term Z and all finite trees

$$h(t_0, \dots, t_{k-1}), \quad t_i \in K_d(X), \quad h \in K_{d-1}(X) \cup \{\text{CAP}_d\}.$$

CAP is greater than every ordinary head. Children remain at the same level; entering a head tree lowers the level. Every individual expression is finite.

At a positive level, use lexicographic path order (LPO) with **(head, arity)** precedence. Its executable recursive comparison is:

1. Z is least. At level zero, compare ROOT/SELF addresses directly.
2. If some immediate child of a is at least b , then a is greater than b ; check the symmetric case next.
3. If neither child case applies, compare heads, then arities, then the first unequal pair of children.

Comparing ordinary heads recursively uses the preceding level. Recursion enters original proper subtrees or lower-level heads; it does not run fundamental sequences. **Arity precedence is essential:** arbitrary arities must not be treated as variants of a single fixed-precedence symbol in this well-ordering argument.

A profile is a pair (d, t) , compared by level first, then by its same-level tree. Different levels are not identified. This is the disjoint ordinal-sum order

$$\mathcal{P}(X) = \sum_{d < \omega} K_d(X).$$

Normalize each column by retaining only the greatest profile at each parent, and sorting parents in decreasing order. Compare columns lexicographically by **(parent, profile)**, with a proper prefix smaller. Compare diagrams left to right by columns. The **controller uses a different priority**: it is the greatest edge in the last column under **(profile, parent)**. These two orders must not be exchanged.

3. Internal seeds and profile lowering

For a block index b define

$$q_{0,b}(p, j) = \begin{cases} p, & b = 0, \\ j, & b > 0; \end{cases}$$

$$q_{d,b}(p, j) = \begin{cases} Z, & b = 0, \\ \text{CAP}_d(Z^{b-1}), & b > 0 \end{cases} \quad (d > 0).$$

Using parent p rather than a fixed ROOT0 at level zero makes these seeds compatible with strictly increasing column relocation. Here b indexes profile approximation; it is **not** the whole diagram's zeroth fundamental-sequence operation.

Write the internal lowering as $\ell_{d,b}$. At level zero, take the finite alphabet predecessor: SELF becomes p , a positive ROOT q becomes $q-1$, and ROOT0 returns no internal predecessor. At a positive level, Z also returns none.

For a nonzero tree $t = h(t_0, \dots, t_{k-1})$:

1. If h is CAP, set the smaller head to $q_{d-1,b}$. Otherwise recursively lower the original head, possibly obtaining no smaller head.
2. Initialize M to the maximum of the same-head leaf and all immediate children when $k > 0$; for $k = 0$, initialize M to Z .
3. The smaller-symbol candidates are $(h, k-1)$ when $k > 0$, and (h', b) if the smaller head h' exists.
4. Recursively lower every nonzero original child, obtaining $u_i = \ell_{d,b}(t_i)$.
5. Perform exactly $b+1$ rounds. Each round replaces M by the maximum of M itself, each smaller symbol applied to its arity-many copies of M , and every available same-head candidate

$$h(t_0, \dots, t_{i-1}, u_i, M, \dots, M).$$

Only original children and the original head are recursively lowered. Newly generated terms are not recursively expanded again in that call, so a single operation is finite. Python uses all child positions. NER uses an exact pruning to the rightmost nonzero position before the final child, plus the final child if nonzero. The paper proves that these yield the same maximum; keeping just one position would not be justified.

The whole-profile lowering $L_b(d, t)$ stays at level d when internal lowering succeeds. If it fails and $d > 0$, use $(d-1, q_{d-1,b})$. Only lowering the bottom-level ROOT0 deletes the edge.

4. Fundamental sequences of diagrams

Zero remains zero. For a nonempty diagram $G = (C_0, \dots, C_x)$:

- $G[0]$ deletes the last column.
- If the last column is empty, every $G[n]$ deletes it.
- Otherwise let c be the controller's parent and $L = x - c$. Put

$$\phi_b(i) = \begin{cases} i, & i < c, \\ i + bL, & i \geq c. \end{cases}$$

The result $G[n]$ has $x + nL$ columns:

1. Keep the columns before the cut once; relocate the source prefix $C_c, \dots, C_{x-1}$ into blocks numbered $b = 0, \dots, n$.
2. The seam for $b < n$ is at $\phi_b(x)$. Relocate every old last-column edge. Retain non-controller edges and apply L_b to the controller **in the destination alphabet**.
3. The seam coincides with the next block's source-cut column. Merge them and retain the maximum profile at each parent.

Parent, child, and all nested ROOT/SELF references use the same map. **Relocate first, then lower** is part of the definition; the operations cannot silently be exchanged. For example, a moved ROOT0 need no longer be ROOT0.

All columns before the old last column remain unchanged, and $G[n]$ is a full-column prefix of $G[n + 1]$. At the first difference, the controller profile at parent c decreases, while new source edges have parents below c . Hence each expansion of a finite nonzero standard term decreases the column order.

In this IPD script, FS, FS_alter, and FS_short implement the same rule. This is not a claim identifying the different wY expansion conventions.

5. TOP, the standard domain, and exact counts

Let A_n have two columns, with its only edge going to parent zero and carrying the minimum level- n profile:

$$A_n = [\emptyset][(0, (n, \min K_n))], \quad \text{Top}[n] = A_n.$$

The minimum is Z at positive levels and ROOT0 at level zero. Standard finite terms are exactly the finite expansion descendants of all these seeds; adjoin an external greatest TOP. TOP's fundamental-sequence terms need not be prefixes of one another. Directly,

$$A_{n+1}[1] = A_n.$$

To count a column, freeze its left prefix, perform a positive-index expansion, and discard newly appended columns. Repeat until the old final column disappears, including the final deletion of an empty column. Thus an empty column has count one, and any positive expansion decreases the old final column's count by exactly one. This is its actual clearing count, not an arbitrary structural code.

$$\text{count}(A_n) = (1, n + 2).$$

Seed count	Counts of its sixth fundamental-sequence term
1,2	1,1,2,4,8,16,32
1,3	1,2,5,13,33,81,193
1,4	1,3,11,44,185,877,7158
1,5	1,4,15,54,190,671,2505
1,6	1,5,21,81,298,1077,3988

For a fixed column there are only finitely many parent positions. The local operation decreases the finite lexicographic product of their profile well-orders, read in decreasing parent order. Thus the count is finite. This

uses profile well-ordering, not the subsequent whole-diagram reflection argument.

No globally exponential upper bound on count growth is claimed. The display uses BigInt, shared trees, exact closed forms and cached barrier subproblems, falling back to the generic algorithm. A budget message such as 1, 3, 11, ... (预算内未算完) reports unfinished computation, not infinity, a 64-bit overflow, or an approximate number.

6. Well-ordering proof outline

The complete paper connects the actual zero-start rule to the standard column order. The final ordinary-Lean theorem has no assumed reflection, supply, or IPD well-ordering premise.

Profiles. Do not assume Kruskal. Given ordinal ranks for the heads, induction on head precedence and already-ranked argument vectors, followed by finite structural induction on a smaller tree, shows that constructors preserve the existence of canonical closed-initial-segment ranks. KP's Σ_1 -Collection joins these ranks. Iteration through the finite levels and a disjoint sum produce an actual set rank for $\mathcal{P}(X)$. Ordinary Lean separately proves all-tree well-foundedness.

Lowering. The actual L_b strictly decreases and preserves ROOT/SELF validity. Increasing maps on the finite alphabet preserve and reflect comparison, including references inside heads. Lowering acts after relocation. This proof does not assume cofinality, index monotonicity, or a general relocation/lowering inequality.

Whole diagrams. In the paper, temporarily work in the constructible inner class and choose its least uncountable κ ; ordinary Lean uses ω_1 . The actual rank of $\mathcal{P}(\kappa + 1)$ supports recursion on (right endpoint, profile, parent endpoint), defining the finite-demand relation R. Equal-profile, smaller-parent requests are allowed; only same-parent profile weakening follows directly from inclusion of requests.

Define coordinate operations of actual least failed requests for each finite tree template and actual least extensions for each finite graph request, fixing only the pre-cut prefix. The operator set is countable. Together with uniform enumerations, finite operation terms generate a countable downward-closed ordinal below κ . This supplies endpoint agreement, every legal profile, and initial representations for arbitrary finite diagrams, without a new reflection or large-cardinal axiom.

The block splice preserves every nested ROOT/SELF source-label identity and represents each seam below the old last label. The least representable last label gives an actual descending ordinal rank. Prefix-related fundamental sequences and zeroth-term deletion make each seed's descendant cone linear under reachability, in agreement with the column order. One-step seed nesting joins the cones. Section 12 of the paper gives the explicit witness closure used by Lean; section 9 transfers the actual rank out of the constructible inner class.

Scope: strict expansion is well-founded on all structurally valid auxiliary graphs. Column-order well-ordering is asserted only for the standard reachable domain. Its columns are canonical; arbitrary raw edge lists need not satisfy the column-decrease premise. One lexicographic decrease alone would not prove global well-ordering.

7. Resource guards and reproduction

NER defaults to about 0.9 seconds per structural event, a separate 0.18-second count budget, at most 4096 columns, 60000 shared tree nodes, 800000 child pointers, and recursion depth 160. Counts additionally have a 65536-bit guard and a display-length guard. Constants are collected in LIMITS. Guard failure neither changes the term nor returns a truncated fundamental-sequence result.

The readable Python file has 249 lines including comments, blanks, validation, local operations and basic printing. The 14 functions implementing construction, comparison and fundamental sequences occupy 143 complete function lines. The longer JS also contains sharing, parsing, counting optimizations, guards and the NER interface; it is not itself a 143-line browser application.

From the repository root, run:

```
python -B tests/test_ipd.py
node --max-old-space-size=256 tests/ipd_display.cjs
```

These bounded tests require no original research workspace, NER source checkout or network. They check the mathematical core, Python/JS expansions and comparisons, exact local counts, and complete tree displays. They are not substitutes for general proofs.

The Python module uses only the standard library. From notations/IPD/:

```
from ipd import IPD, TOP
a = IPD.seed(2)
print(a)          # [[0:2/.]
print(a.fs(3))    # [[0:1/.][1:1/^[2:1/^(.)]
assert TOP.fs(2) == a
assert IPD.seed(3).fs(1) == a
assert a.fs(2).columns == a.fs(3).columns[:len(a.fs(2).columns)]
```

Python has no overall resource cap; callers can provide a tick hook to interrupt expensive evaluation. NER has the event limits above. Both original implementations are byte-preserved; historical embedded proof-status notes are superseded by this definition and the release verification record.